

443_Project

ANURAG NEPAL

2025-12-16

```
# Load necessary libraries
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
```

```
##   method      from
```

```
##   as.zoo.data.frame zoo
```

```
library(tseries)
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##   filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   intersect, setdiff, setequal, union
```

```
library(lubridate)
```

```
##
```

```
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   date, intersect, setdiff, union
```

```
library(knitr)
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
library(astsa)
```

```
##  
## Attaching package: 'astsa'
```

```
## The following object is masked from 'package:forecast':  
##  
##     gas
```

```
library(itsmr)
```

```
##  
## Attaching package: 'itsmr'
```

```
## The following object is masked from 'package:astsa':  
##  
##     trend
```

```
## The following object is masked from 'package:tseries':  
##  
##     arma
```

```
## The following object is masked from 'package:forecast':  
##  
##     forecast
```

```
library(ltsa)  
library(car)
```

```
## Loading required package: carData
```

```
##  
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':  
##  
##     recode
```

```
# =====  
# DATA LOADING & CLEANING  
# =====  
# Load Data  
raw_data <- read.csv("/Users/anurag/Downloads/National_Custom_Data.csv", stringsAsFactors = FALSE)  
  
# Clean Numerical Columns (Remove commas)  
clean_col <- function(x) as.numeric(gsub(",", "", x))  
raw_data$NUM.INFLUENZA.DEATHS <- clean_col(raw_data$NUM.INFLUENZA.DEATHS)  
raw_data$NUM.PNEUMONIA.DEATHS <- clean_col(raw_data$NUM.PNEUMONIA.DEATHS)  
raw_data$TOTAL.DEATHS <- clean_col(raw_data$TOTAL.DEATHS)  
  
# Create Date Column  
get_cal_year <- function(season, week) {
```

```

    start_year <- as.numeric(substr(season, 1, 4))
    if (week >= 40) return(start_year)
    else return(start_year + 1)
}
raw_data$Year <- mapply(get_cal_year, raw_data$SEASON, raw_data$WEEK)
raw_data$Date <- as.Date(paste(raw_data$Year, raw_data$WEEK, 1, sep="-"), format="%Y-%U-%u")

## Warning in strptime(x, format, tz = "GMT"): (0-based) yday 369 in year 2020 is
## invalid
## Warning in strptime(x, format, tz = "GMT"): (0-based) yday 369 in year 2020 is
## invalid

# Aggregate to Monthly Level
monthly_data <- raw_data %>%
  group_by(MonthDate = floor_date(Date, "month")) %>%
  summarise(
    Total_P_I = sum(NUM.INFLUENZA.DEATHS + NUM.PNEUMONIA.DEATHS, na.rm = TRUE),
    Total_All_Deaths = sum(TOTAL.DEATHS, na.rm = TRUE)
  ) %>%
  mutate(Percent_PI = (Total_P_I / Total_All_Deaths) * 100) %>%
  arrange(MonthDate) %>%
  filter(!is.na(MonthDate))

# Create Time Series Objects
start_yr <- year(min(monthly_data$MonthDate))
start_mo <- month(min(monthly_data$MonthDate))

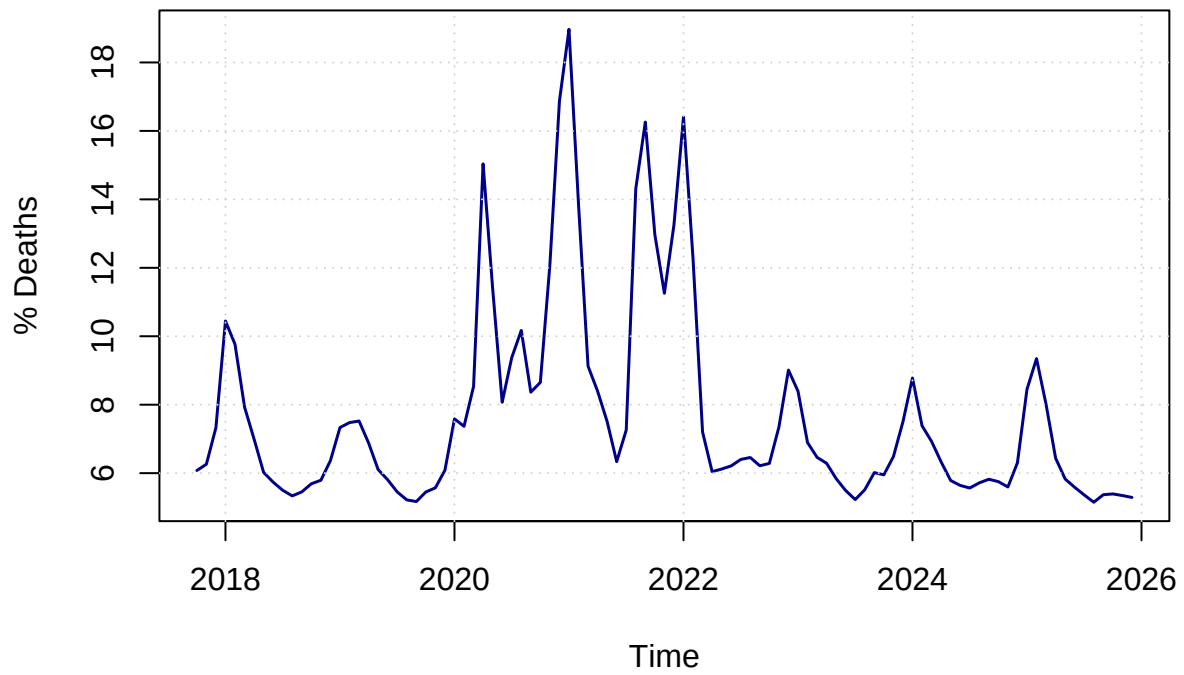
# 1. Raw Data (Includes COVID Spikes)
ts_raw <- ts(monthly_data$Percent_PI, start = c(start_yr, start_mo), frequency = 12)

# 2. Cleaned Data (Outliers Interpolated for Robust Modeling)
ts_cleaned <- tsclean(ts_raw)

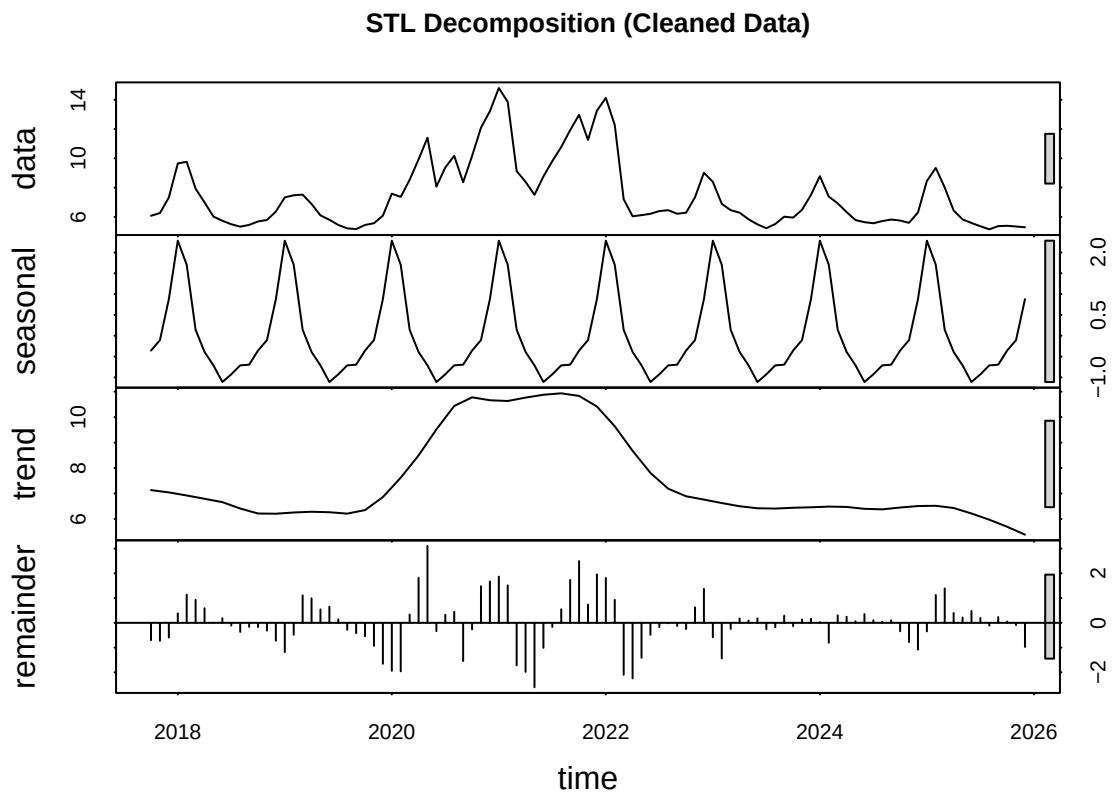
# Plot Raw Data
plot(ts_raw, main="US P&I Mortality Rate (Raw Data)", ylab="% Deaths", col="darkblue", lwd=1.5)
grid()

```

US P&I Mortality Rate (Raw Data)

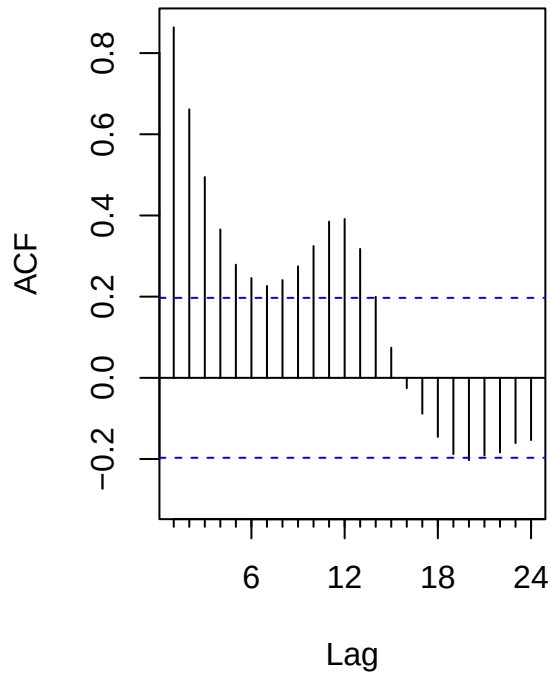


```
# STL Decomposition (Using Cleaned Data to see clear seasonality)
decomp <- stl(ts_cleaned, s.window="periodic")
plot(decomp, main="STL Decomposition (Cleaned Data)")
```

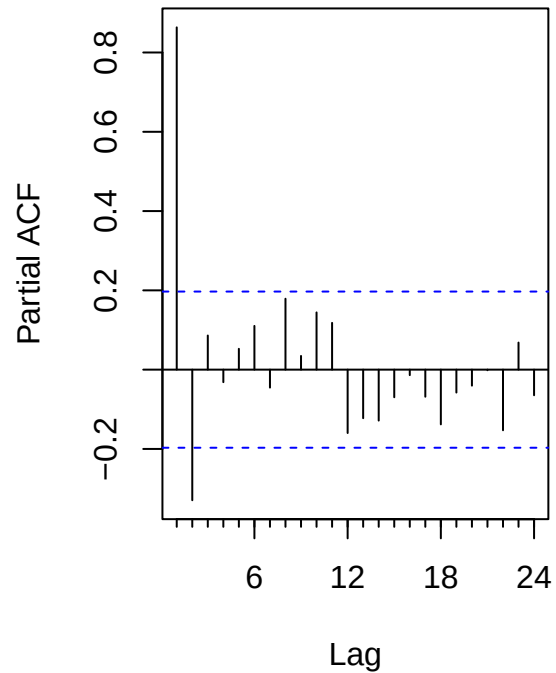


```
# ACF and PACF
par(mfrow=c(1,2))
Acf(ts_cleaned, main="ACF (Cleaned Data)")
Pacf(ts_cleaned, main="PACF (Cleaned Data)")
```

ACF (Cleaned Data)



PACF (Cleaned Data)



```
par(mfrow=c(1,1))
```

```
# 1. Test on Level Data
print("--- ADF Test: Level ---")
```

```
## [1] "--- ADF Test: Level ---"
```

```
print(aTSA::adf.test(ts_cleaned))
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag      ADF p.value
## [1,]  0 -0.803   0.391
## [2,]  1 -1.026   0.310
## [3,]  2 -0.926   0.346
## [4,]  3 -1.067   0.295
## Type 2: with drift no trend
##      lag      ADF p.value
## [1,]  0 -2.54   0.1188
## [2,]  1 -3.50   0.0103
## [3,]  2 -2.91   0.0495
## [4,]  3 -2.87   0.0555
```

```
## Type 3: with drift and trend
##      lag    ADF p.value
## [1,]  0 -2.66  0.3015
## [2,]  1 -3.61  0.0361
## [3,]  2 -3.00  0.1607
## [4,]  3 -2.91  0.1998
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag    ADF    p.value
## [1,]  0 -0.8034741 0.3905352
## [2,]  1 -1.0262195 0.3102956
## [3,]  2 -0.9258934 0.3464361
## [4,]  3 -1.0674007 0.2954608
##
## $type2
##      lag    ADF    p.value
## [1,]  0 -2.541870 0.11883178
## [2,]  1 -3.497098 0.01029448
## [3,]  2 -2.906392 0.04952136
## [4,]  3 -2.867271 0.05545830
##
## $type3
##      lag    ADF    p.value
## [1,]  0 -2.656604 0.30154692
## [2,]  1 -3.607885 0.03612138
## [3,]  2 -3.002233 0.16070374
## [4,]  3 -2.906207 0.19983410
```

```
# 2. Test on Seasonal Difference (D=1)
ts_diff <- diff(ts_cleaned, lag=12)
print("--- ADF Test: Seasonal Difference (D=1) ---")
```

```
## [1] "--- ADF Test: Seasonal Difference (D=1) ---"
```

```
print(aTSA::adf.test(ts_diff))
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag    ADF p.value
## [1,]  0 -2.66  0.0100
## [2,]  1 -2.93  0.0100
## [3,]  2 -2.50  0.0143
## [4,]  3 -2.54  0.0125
## Type 2: with drift no trend
##      lag    ADF p.value
## [1,]  0 -2.65  0.0914
## [2,]  1 -2.91  0.0499
## [3,]  2 -2.49  0.1409
## [4,]  3 -2.53  0.1255
## Type 3: with drift and trend
```

```
##      lag    ADF p.value
## [1,]  0 -2.72  0.275
## [2,]  1 -3.00  0.162
## [3,]  2 -2.60  0.324
## [4,]  3 -2.71  0.280
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag      ADF    p.value
## [1,]  0 -2.660852 0.01000000
## [2,]  1 -2.925013 0.01000000
## [3,]  2 -2.501546 0.01430213
## [4,]  3 -2.544426 0.01252926
##
## $type2
##      lag      ADF    p.value
## [1,]  0 -2.647268 0.09137071
## [2,]  1 -2.909390 0.04991106
## [3,]  2 -2.486887 0.14090899
## [4,]  3 -2.526822 0.12548383
##
## $type3
##      lag      ADF    p.value
## [1,]  0 -2.724249 0.2752020
## [2,]  1 -3.004827 0.1619747
## [3,]  2 -2.604446 0.3235489
## [4,]  3 -2.712112 0.2801000
```

```
# 3. Test on Double Difference (D=1, d=1)
# Often required if seasonal differencing alone doesn't remove the trend
ts_doubly_diff <- diff(ts_diff, lag=1)
print("--- ADF Test: Seasonal + First Difference (D=1, d=1) ---")
```

```
## [1] "--- ADF Test: Seasonal + First Difference (D=1, d=1) ---"
```

```
print(aTSA::adf.test(ts_doubly_diff))
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag    ADF p.value
## [1,]  0 -8.65  0.01
## [2,]  1 -7.48  0.01
## [3,]  2 -5.90  0.01
## [4,]  3 -5.33  0.01
## Type 2: with drift no trend
##      lag    ADF p.value
## [1,]  0 -8.59  0.01
## [2,]  1 -7.43  0.01
## [3,]  2 -5.86  0.01
## [4,]  3 -5.30  0.01
## Type 3: with drift and trend
```

```
##      lag    ADF p.value
## [1,]  0 -8.54   0.01
## [2,]  1 -7.39   0.01
## [3,]  2 -5.84   0.01
## [4,]  3 -5.28   0.01
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag      ADF p.value
## [1,]  0 -8.645996   0.01
## [2,]  1 -7.476852   0.01
## [3,]  2 -5.899900   0.01
## [4,]  3 -5.331257   0.01
##
## $type2
##      lag      ADF p.value
## [1,]  0 -8.594412   0.01
## [2,]  1 -7.431110   0.01
## [3,]  2 -5.863559   0.01
## [4,]  3 -5.297977   0.01
##
## $type3
##      lag      ADF p.value
## [1,]  0 -8.544893   0.01
## [2,]  1 -7.390294   0.01
## [3,]  2 -5.840880   0.01
## [4,]  3 -5.283964   0.01
```

```
# Model 1: ARIMA on Raw Data
```

```
fit_raw <- auto.arima(ts_raw, seasonal=TRUE, stepwise=FALSE, approximation=FALSE)
```

```
# Model 2: ARIMA on Cleaned Data (No Transformation)
```

```
# We force D=1 based on EDA/Stationarity tests
```

```
fit_clean <- auto.arima(ts_cleaned, seasonal=TRUE, D=1, stepwise=FALSE, approximation=FALSE)
```

```
fit_log_raw <- auto.arima(ts_raw, seasonal=TRUE, D=1, lambda=0, stepwise=FALSE, approximation=FALSE)
```

```
# Model 3: ARIMA on Cleaned Data + Log Transformation (Robust)
```

```
fit_log_clean <- auto.arima(ts_cleaned, seasonal=TRUE, D=1, lambda=0, stepwise=FALSE, approximation=FALSE)
```

```
# Function to extract metrics
```

```
get_accuracy <- function(model, name) {
  acc <- accuracy(model)
  return(data.frame(
    Model = name,
    AICc = round(model$aicc, 2),
    RMSE = round(acc[2], 4),
    MAPE = round(acc[5], 4)
  ))
}
```

```
# Combine into a table
```

```
comparison_table <- rbind(
  get_accuracy(fit_raw, "ARIMA (Raw Data)"),
```

```

get_accuracy(fit_clean, "ARIMA (Cleaned Data)",
  get_accuracy(fit_log_raw, "ARIMA (Raw + Log)",
    get_accuracy(fit_log_clean, "ARIMA (Cleaned + Log)"
  )
)

kable(comparison_table, caption = "Model Accuracy Comparison")

```

Table 1: Model Accuracy Comparison

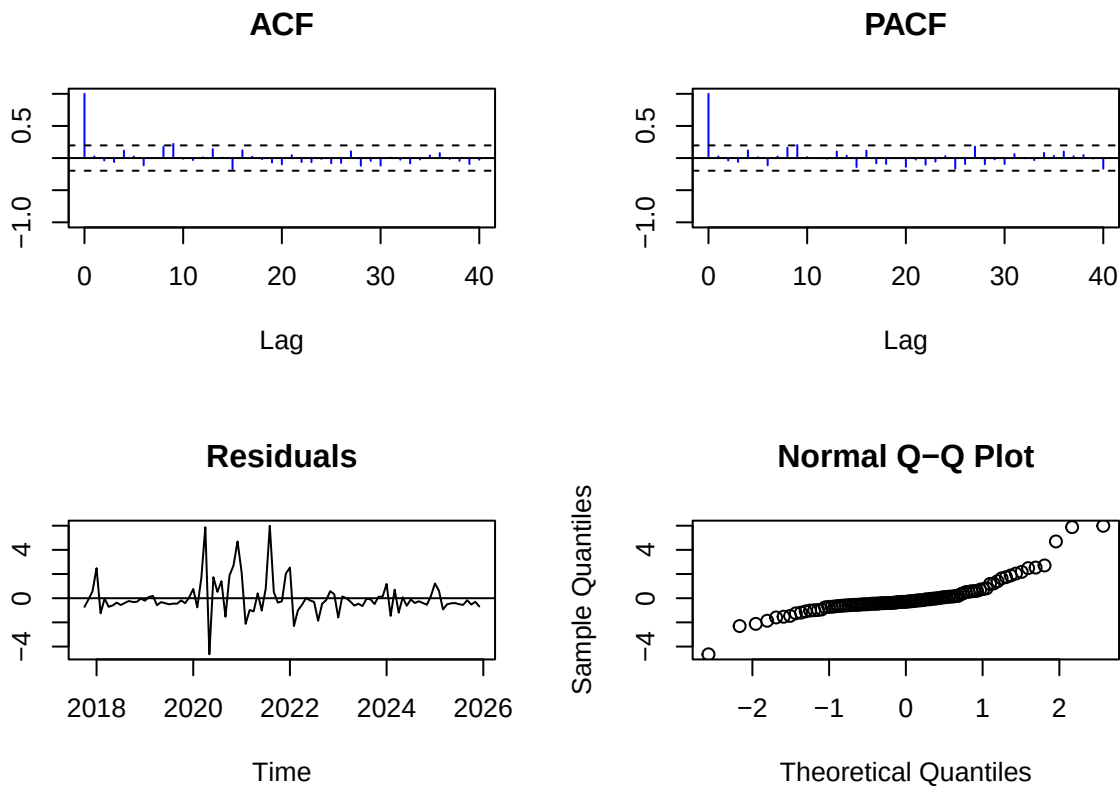
Model	AICc	RMSE	MAPE
ARIMA (Raw Data)	363.24	1.4169	9.9286
ARIMA (Cleaned Data)	291.47	1.0927	9.7174
ARIMA (Raw + Log)	-72.83	1.3309	9.0446
ARIMA (Cleaned + Log)	-96.64	1.0127	8.3257

```
test(fit_raw$residuals)
```

```

## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q   Q ~ chisq(20)    21.66   0.3593
## McLeod-Li Q   Q ~ chisq(20)    47.23   5e-04 *
## Turning points T (T-64.7)/4.2 ~ N(0,1)    63   0.6884
## Diff signs S   (S-49)/2.9 ~ N(0,1)    54   0.0833
## Rank P        (P-2425.5)/165.4 ~ N(0,1)   2317  0.5118

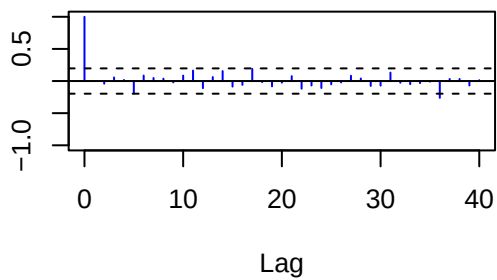
```



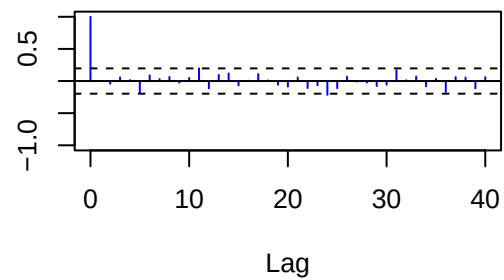
```
test(fit_clean$residuals)
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)    20.89    0.4035
## McLeod-Li Q    Q ~ chisq(20)    53.03    1e-04 *
## Turning points T (T-64.7)/4.2 ~ N(0,1)    59    0.1728
## Diff signs S    (S-49)/2.9 ~ N(0,1)    42    0.0153 *
## Rank P          (P-2425.5)/165.4 ~ N(0,1)    2014    0.0128 *
```

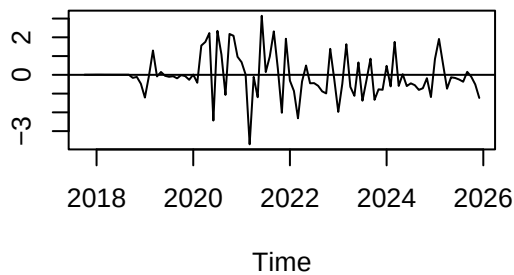
ACF



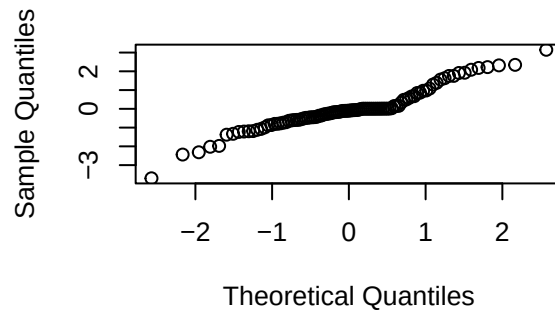
PACF



Residuals



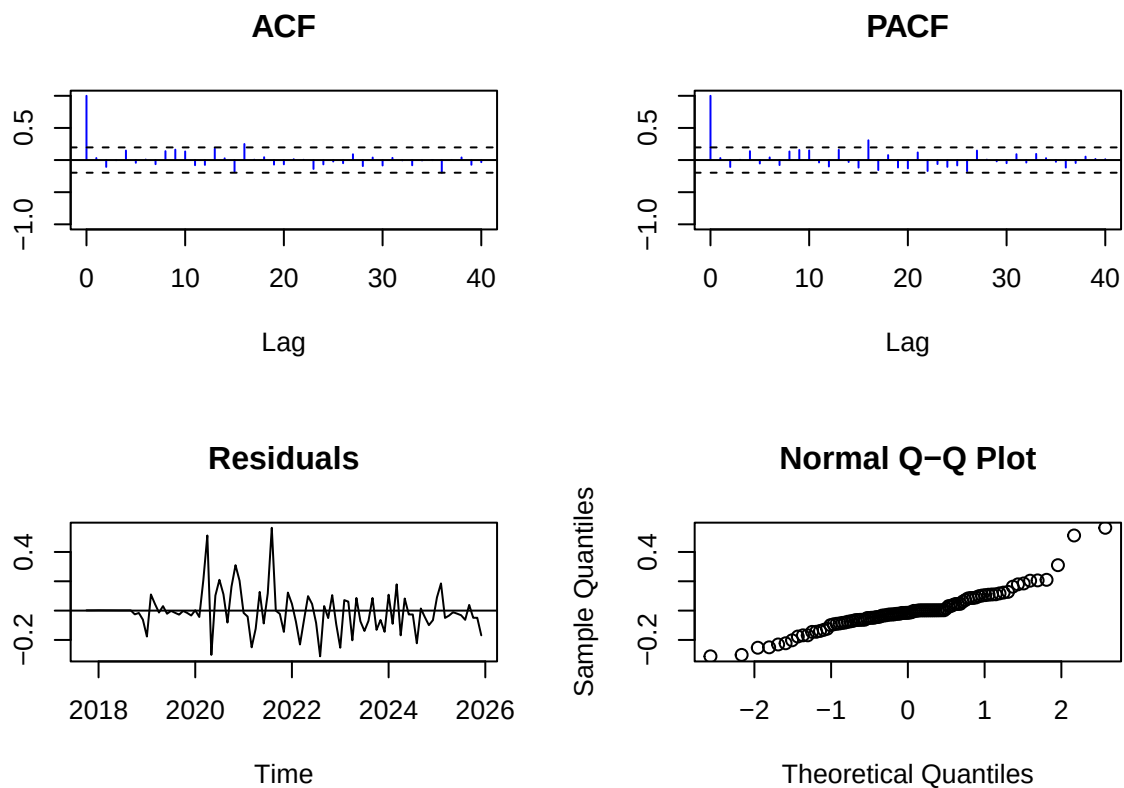
Normal Q-Q Plot



```
# Check Residuals of the Best Model
```

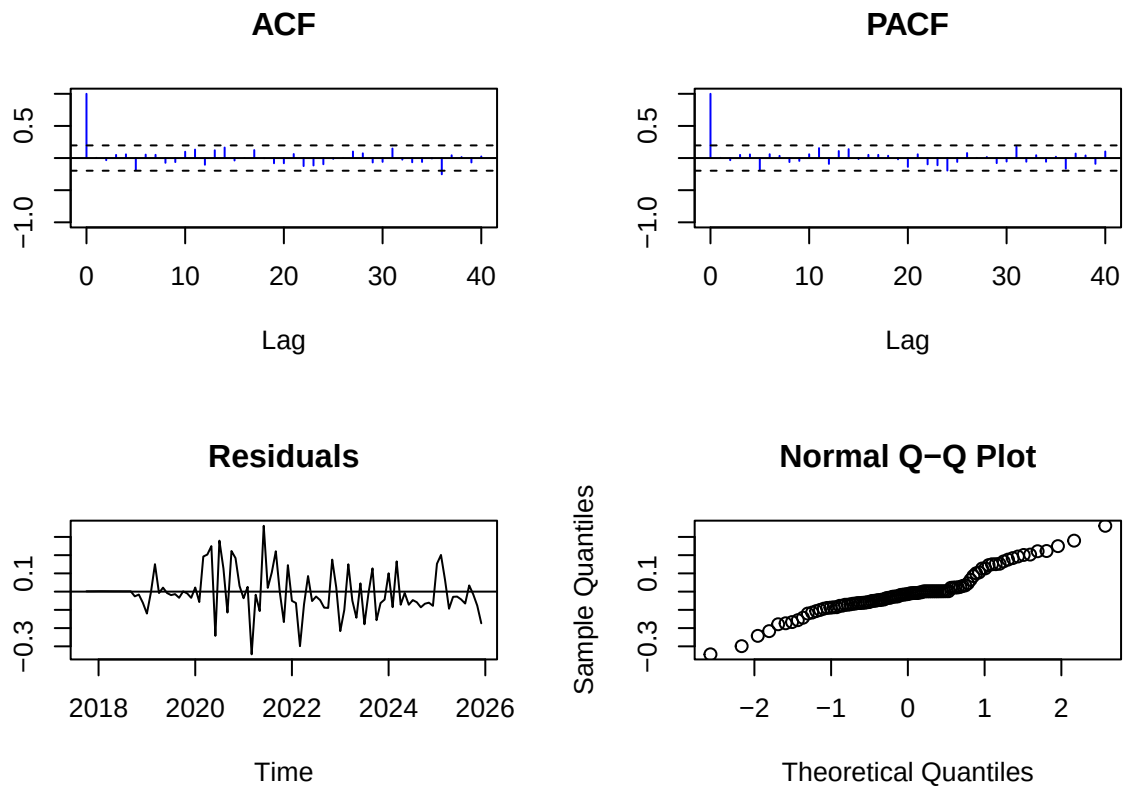
```
test(fit_log_raw$residuals)
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)    29.03    0.0872
## McLeod-Li Q    Q ~ chisq(20)    28.28    0.103
## Turning points T (T-64.7)/4.2 ~ N(0,1)    57    0.0651
## Diff signs S    (S-49)/2.9 ~ N(0,1)    41    0.0056 *
## Rank P          (P-2425.5)/165.4 ~ N(0,1)    2037    0.0188 *
```



```
test(fit_log_clean$residuals)
```

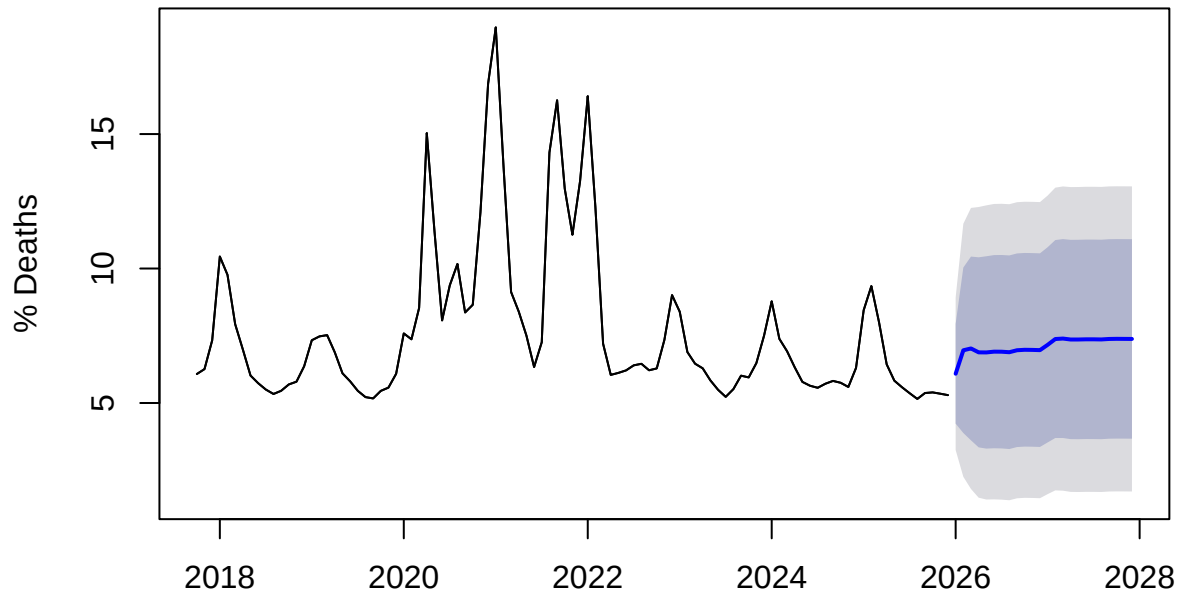
```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic p-value
## Ljung-Box Q    Q ~ chisq(20)    18.67    0.5432
## McLeod-Li Q    Q ~ chisq(20)    38.79    0.0071 *
## Turning points T (T-64.7)/4.2 ~ N(0,1)    59    0.1728
## Diff signs S   (S-49)/2.9 ~ N(0,1)    45    0.1659
## Rank P         (P-2425.5)/165.4 ~ N(0,1) 1979    0.0069 *
```



```
# Generate Forecasts
fc_raw <- forecast::forecast(fit_raw, h=24)
fc_clean <- forecast::forecast(fit_clean, h=24)
fc_log_raw <- forecast::forecast(fit_log_raw, h=24, biasadj=TRUE)
fc_log_clean <- forecast::forecast(fit_log_clean, h=24, biasadj=TRUE)

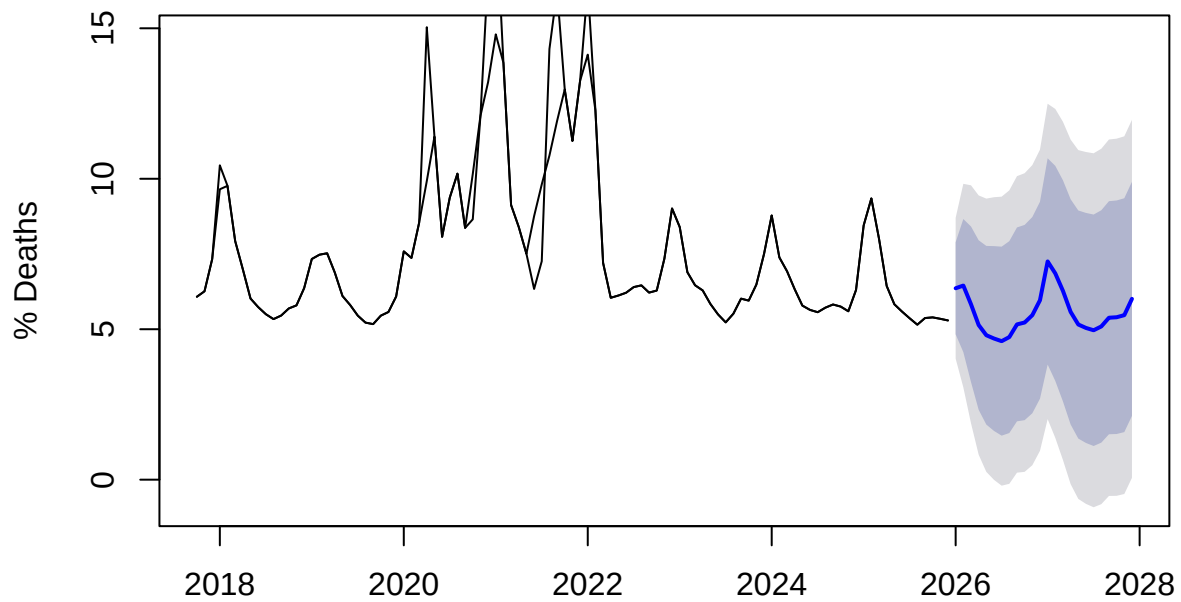
plot(fc_raw, main="Forecast: Raw Model (No Log Transform)", ylab="% Deaths", fcol="blue")
lines(ts_raw, col="black")
```

Forecast: Raw Model (No Log Transform)



```
# PLOT 1: Cleaned Data Model (Often crashes or goes negative)
plot(fc_clean, main="Forecast: Cleaned Model (No Log Transform)", ylab="% Deaths", fcol="blue")
lines(ts_raw, col="black")
```

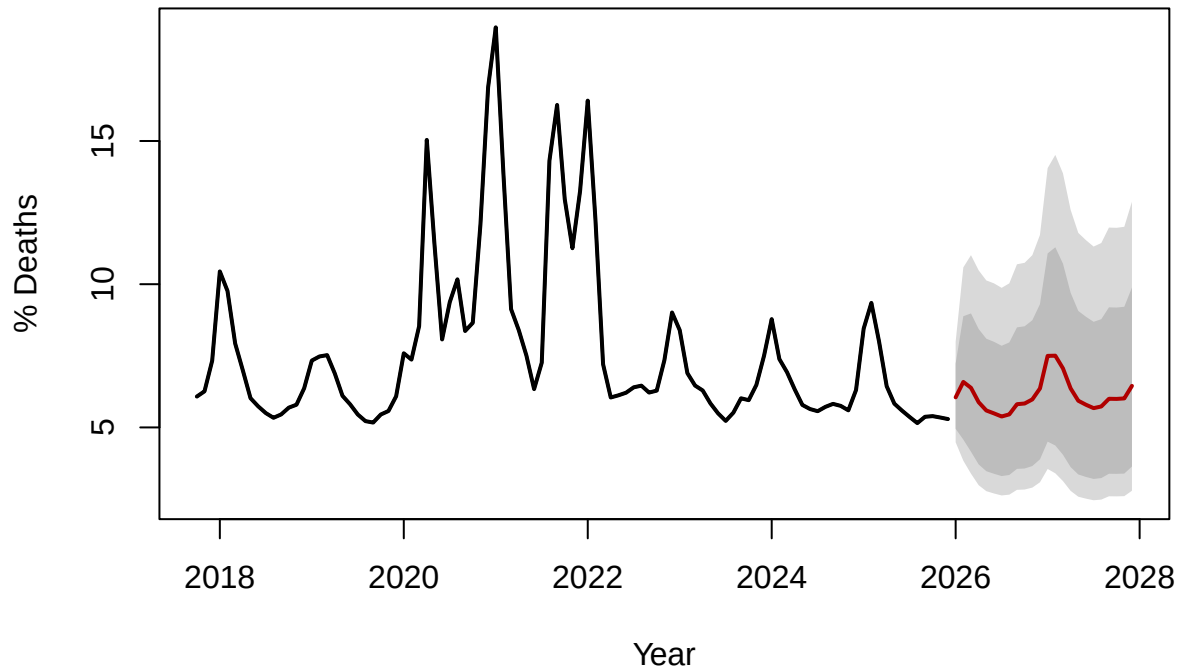
Forecast: Cleaned Model (No Log Transform)



```
# PLOT 2: Log-Transformed Model (Final Selected Model)
plot(fc_log_raw,
     main="24-Month Forecast: US P&I Mortality (Log-Transformed+ RAW)",
     ylab="% Deaths",
     xlab="Year",
     fcol="#b00000",          # Dark Red
     flty=1, lwd=2,
     shadecols=c("#d9d9d9", "#bdbdbd")
)      # Focus on realistic range

# Overlay Historical Data
lines(ts_raw, col="black", lwd=1.5)
```

24-Month Forecast: US P&I Mortality (Log-Transformed+ RAW)

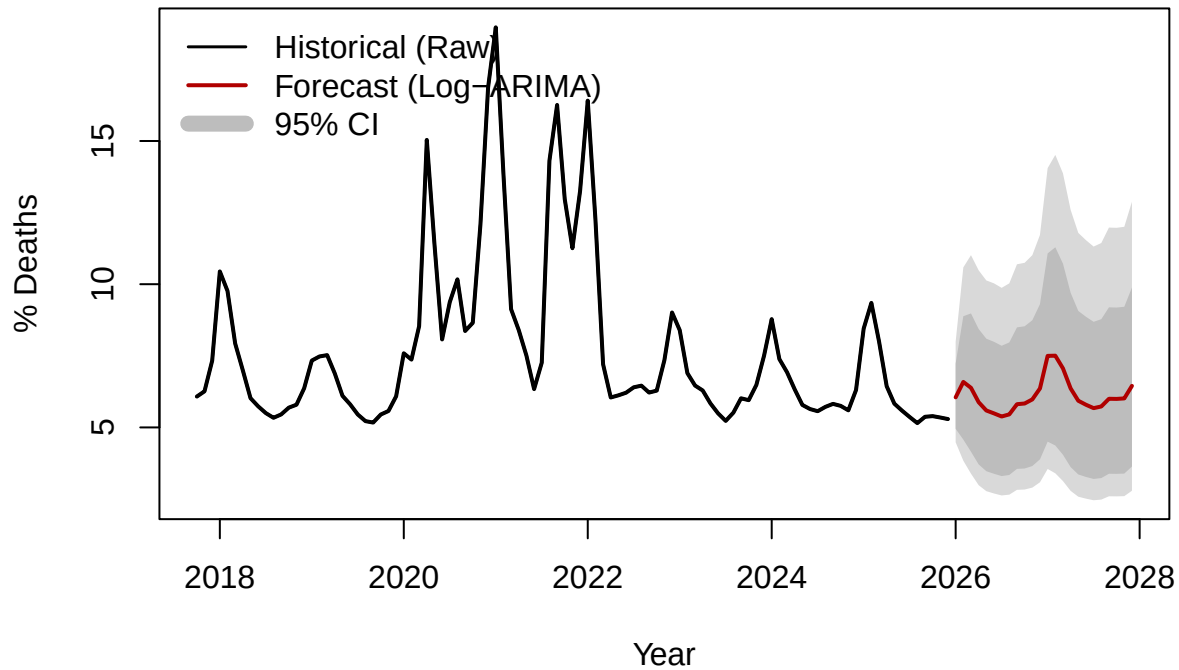


```
plot(fc_log_raw,
     main="24-Month Forecast: US P&I Mortality (Log-Transformed+ Cleaned)",
     ylab="% Deaths",
     xlab="Year",
     fcol="#b00000",          # Dark Red
     flty=1, lwd=2,
     shadecols=c("#d9d9d9", "#bdbdbd")
)                                # Focus on realistic range

# Overlay Historical Data
lines(ts_raw, col="black", lwd=1.5)

legend("topleft",
     legend=c("Historical (Raw)", "Forecast (Log-ARIMA)", "95% CI"),
     col=c("black", "#b00000", "#bdbdbd"),
     lty=c(1, 1, 1), lwd=c(1.5, 2, 8), bty="n")
```

24-Month Forecast: US P&I Mortality (Log-Transformed+ Cleaned)



```
# Extract forecast values into a readable dataframe
fc_values <- data.frame(
  Date = seq(from = max(monthly_data$MonthDate) %m+% months(1), by = "month", length.out = 24),
  Forecast = round(fc_log_clean$mean, 2),
  Lower_95 = round(fc_log_clean$lower[, "95%"], 2),
  Upper_95 = round(fc_log_clean$upper[, "95%"], 2)
)

kable(fc_values, caption = "24-Month Forecast Values (Log-ARIMA)")
```

Table 2: 24-Month Forecast Values (Log-ARIMA)

Date	Forecast	Lower_95	Upper_95
2026-01-01	6.15	4.75	7.84
2026-02-01	6.21	4.23	8.81
2026-03-01	5.81	3.67	8.76
2026-04-01	5.30	3.19	8.31
2026-05-01	5.02	2.91	8.08
2026-06-01	4.91	2.78	8.06
2026-07-01	4.82	2.68	8.02
2026-08-01	4.93	2.71	8.29
2026-09-01	5.32	2.90	9.01
2026-10-01	5.38	2.91	9.16
2026-11-01	5.61	3.01	9.58

Date	Forecast	Lower_95	Upper_95
2026-12-01	6.05	3.24	10.36
2027-01-01	7.20	3.73	12.63
2027-02-01	6.84	3.44	12.29
2027-03-01	6.41	3.15	11.70
2027-04-01	5.82	2.81	10.75
2027-05-01	5.43	2.59	10.13
2027-06-01	5.32	2.52	9.99
2027-07-01	5.25	2.46	9.89
2027-08-01	5.37	2.51	10.15
2027-09-01	5.65	2.63	10.73
2027-10-01	5.67	2.63	10.79
2027-11-01	5.74	2.66	10.92
2027-12-01	6.26	2.89	11.94